

翼型工程数据库系统 (AEDS)

项目不足与缺点分析报告

——基于数据库原理与实践课程考核要求的系统复盘

评估日期：2026 年 6 月 21 日

评估说明

本报告以数据库原理与实践课程的大二软件工程专业学生视角，依据课程核心知识点（数据库设计、事务处理、并发控制、数据完整性、存储优化、SQL 查询性能、数据库安全性、系统开发），对 AEDS 项目的实现质量进行全面复盘。分析聚焦于项目在核心数据库理论落地过程中暴露出的设计缺陷与工程短板，不重复罗列已达标部分。

1 项目整体完成情况总评

在正式进入不足分析之前，有必要客观评价项目的整体完成度。根据《差异检测报告（第 5 版）》的逐项对照，AEDS 系统在 35 项考核指标中实现了 **28 项完全达标 + 3 项部分达标**，综合达标率 **80%**。具体而言：

- §5 系统功能要求**：7/7 项全部达标，CRUD、版本管理、坐标录入、性能录入均已通过 Web 交互实现闭环。
- §7 高级机制实验**：索引优化（5.3x 性能提升）、事务并发（原子回滚 + 乐观锁/悲观锁）、视图/触发器/存储过程均已完成，21/21 项覆盖率检查 100%。
- §8 数据治理**：8 条异常检测规则 + 版本追溯 + 来源标记 + 软删除策略均已实现。
- §10 可视化**：13 张 ECharts 图表 + 6 张 Matplotlib 静态图，远超文档要求的“两项以上”。

然而，在高标准率的表象下，项目中存在若干触及数据库课程核心考核维度的系统性短板。以下三点不足并非表层缺陷，而是从数据库原理的底层视角审视项目时必然浮现的实质性问题。

2 不足一：事务原子性在业务层的落地断层

2.1 问题描述

项目在 TestLab 实验套件中出色地验证了事务的原子回滚与并发隔离机制（见 02_transaction_experimen 但在实际的 Web 业务代码中，事务管理呈现严重的“实验与生产脱节”现象。

2.2 技术证据

Django Webfront 的 DAO 层代码中,涉及多表联合写入的关键操作未使用显式事务包裹。以 `airfoil_dao.py` 中的 `create_airfoil()` 为例:

Listing 1: 多表写入未包裹事务 (`airfoil_dao.py`)

```
1 def create_airfoil(code, name, ...):
2     with connection.cursor() as cursor:          # ← 仅管理游标生命周期
3         # 写入 1: data_source
4         cursor.execute("INSERT INTO data_source (...) VALUES (...)")
5         # 写入 2: airfoil
6         cursor.execute("INSERT INTO airfoil (...) VALUES (...)")
7         # 写入 3: airfoil_version
8         cursor.execute("INSERT INTO airfoil_version (...) VALUES (...)")
```

该函数使用 `with connection.cursor()` 上下文管理器,但 Django 的 `cursor` 上下文仅管理游标(`cursor`)的开闭,不提供事务管理语义。在当前 Django 配置下(`ATOMIC_REQUESTS=False`, 即默认值),每个 `cursor.execute()` 在 `autocommit` 模式下独立提交。这意味着:

- 若 `airfoil` 的 `INSERT` 成功但 `airfoil_version` 的 `INSERT` 失败(如版本号冲突), `data_source` 记录将成为孤儿数据(`orphan record`)。
- 系统失去了“要么三表同时写入,要么全部不写入”的原子性保证。

在整个 Webfront 代码库中,仅 `anomaly_engine.py` 的批量扫描函数使用了 `transaction.atomic()`,而存在多表写入风险的操作(新建翼型、新建版本、新建性能数据——均涉及后台多 `INSERT` 步骤)全部依赖数据库的 `autocommit` 机制。

2.3 课程知识映射

数据库原理课程的“事务”一章明确指出事务的 ACID 属性是数据库区别于文件系统的核心特征。特别是原子性(**Atomicity**)要求:一个事务中的所有操作要么全部完成,要么全部不执行。当前系统的实现方式使得这一点在理论上被 TestLab 验证、在实际上被生产代码绕过,构成典型的“知识点掌握不完整”问题。

2.4 修复思路

在 `create_airfoil`、`create_version`、`add_performance_record` 等涉及多表写入的 DAO 函数中,添加 `with transaction.atomic():` 包装整个写入逻辑块。

3 不足二: 完整性约束设计的策略性妥协

3.1 问题描述

项目在物理层 CHECK 约束的设计上存在一个值得从数据库原理角度深入审视的构造性缺陷:将业务语义标记(`is_anomaly`)作为数据完整性约束的“豁免条件”,导致约束不再是绝对不可违反的物理规则。

3.2 技术证据

performance_record 表上 Cd 约束的实际定义（见 02_constraints.sql）:

Listing 2: 条件化 CHECK 约束（02_constraints.sql）

```
1 ALTER TABLE public.performance_record
2   ADD CONSTRAINT ck_performance_record_cd_or_anomaly
3   CHECK (cd >= 0 OR is_anomaly);
```

该约束的逻辑语义是：“Cd 必须大于等于 0，除非该记录被标记为异常”。这在工程上是务实的——它允许异常检测引擎将识别到的负 Cd 记录标注为异常而非直接拒绝写入。但从数据库原理的角度审视：

1. **约束不再是约束：**CHECK 的原始设计意图是“物理不可违反”，而此处的 is_anomaly 参数使得任何能够设置 is_anomaly=true 的应用程序都可以绕过物理层的 Cd 约束。约束的执行权从 DBMS 转移到了应用层。
2. **语义混乱：**is_anomaly 是一个业务元数据标记（“这条数据疑似有问题”），却被用作物理约束的豁免条件（“因为标注了异常，所以允许违反物理定律”）。这两层语义的目的不同，强行耦合引入了概念上的自相矛盾。
3. **课程标准对照：**文档 §6.3 明确要求“阻力系数 Cd 必须大于等于 0”。当前实现添加了 OR is_anomaly 后缀，严格来说只能算“Cd 大于等于 0 或被标记异常”，未完全满足文档的本意。

此外，experiment_condition 表缺少 (alpha_deg, reynolds_number) 的唯一性约束，理论上允许同一工况组合在表中以不同 UUID 重复出现。虽然 performance_record 的联合唯一约束 (version_id, condition_id) 能间接避免重复性能记录，但物理层面没有防止工况参数表自身的冗余。

3.3 课程知识映射

数据库原理课程的“完整性”一章将约束分为实体完整性、参照完整性、用户定义完整性三类。CHECK 属于用户定义完整性，其设计原则是：约束应在数据库层强制执行，不应依赖应用层的“自觉”。当约束条件中引入可由应用层修改的字段作为豁免条件时，约束的物理强制性已经退化。

3.4 修复思路

- 将 Cd 约束恢复为纯物理约束 CHECK (cd >= 0)，异常数据改为在应用层通过插入前校验捕获——不让不合规数据落入数据库，而非先写入再标记异常。
- 补充 experiment_condition 表上的 UNIQUE (alpha_deg, reynolds_number) 约束。

4 不足三：数据库安全性与访问控制的体系性缺失

4.1 问题描述

文档 §6.1 要求至少包含 `User` (用户或操作人员) 实体, 项目在表结构上实现了 `user_account` 表并建立了外键关联。但从数据库安全性这一课程核心知识点的视角审视, 系统在访问控制、身份认证和权限管理方面存在系统性空白。

4.2 技术证据

1. 表设计层面: `user_account` 表仅包含 `username` 和 `role` 两个字段, 缺少 `password_hash` ——这意味着表中存储的是裸标识而非可认证的凭据。外键引用 (如 `airfoil_version.created_by` → `user_account.user_id`) 更多地起到”操作人记录”作用而非”访问控制”作用 (见表 1)。

表 1: `user_account` 表结构分析

字段	类型	评估
<code>user_id</code>	UUID PK	符合规范
<code>username</code>	TEXT	基本标识
<code>role</code>	TEXT	枚举值未约束, 无 CHECK
<code>is_active</code>	BOOLEAN	有标记字段
<code>password_hash</code>	缺失	无法实现认证

2. 数据库层缺失: 系统未实现任何以下 PostgreSQL 数据库安全机制:

- 角色 (Role) 分离: 应用使用单一数据库连接用户, 无法区分读写权限。
- 行级安全 (RLS, Row-Level Security): 未针对 `user_account` 或敏感表启用 RLS 策略, 所有用户可读取所有数据。
- 授权 (GRANT / REVOKE) 管理: 未在初始化脚本中建立按角色的表级权限分配。

3. 应用层退化: Django Web 前端主动移除了登录认证 (注释掉 `LOGIN_URL`, 禁用 `@login_required` 装饰器), 以追求”开箱即用”的易用性。这一设计选择在工程上可以理解 (便于演示), 但在课程考核的”数据库安全性”维度上构成实质性空白。

4. 审计日志的伪安全性: 触发器 `trg_log_change_*` 通过 `current_setting('app.current_user_id')` 记录操作人, 但 `current_setting` 是会话级变量, 可由任意会话通过 `SET app.current_user_id = 'xxx'` 的方式伪造。它提供了可追溯性 (auditability), 但不提供任何访问控制执行能力 (access control enforcement)。

4.3 课程知识映射

数据库原理课程的” 安全性” 专题涵盖：身份认证（Authentication）、授权控制（Authorization）、强制访问控制（MAC）、自主访问控制（DAC）、审计追踪（Audit）。当前系统的 user_account 表仅覆盖了审计追踪的” 事后记录” 层面，而完全缺失身份认证（无密码）和授权控制（无角色权限分级、无 RLS、无 GRANT/REVOKE）这两个更基本的数据库安全维度。

4.4 修复思路

- 最低要求：为 user_account 表增加 password_hash（至少使用 pgcrypto 扩展的 bcrypt 哈希），在应用层实现基于会话的登录认证。
- 课程合规：建立至少两种数据库角色（reader / editor），使用 GRANT SELECT/GRANT INSERT 等语句分配表级权限。
- 进阶：对性能数据等核心表启用 RLS 策略，基于 created_by 字段限制用户只能修改自己创建的数据。

5 系统性短板架构分析图

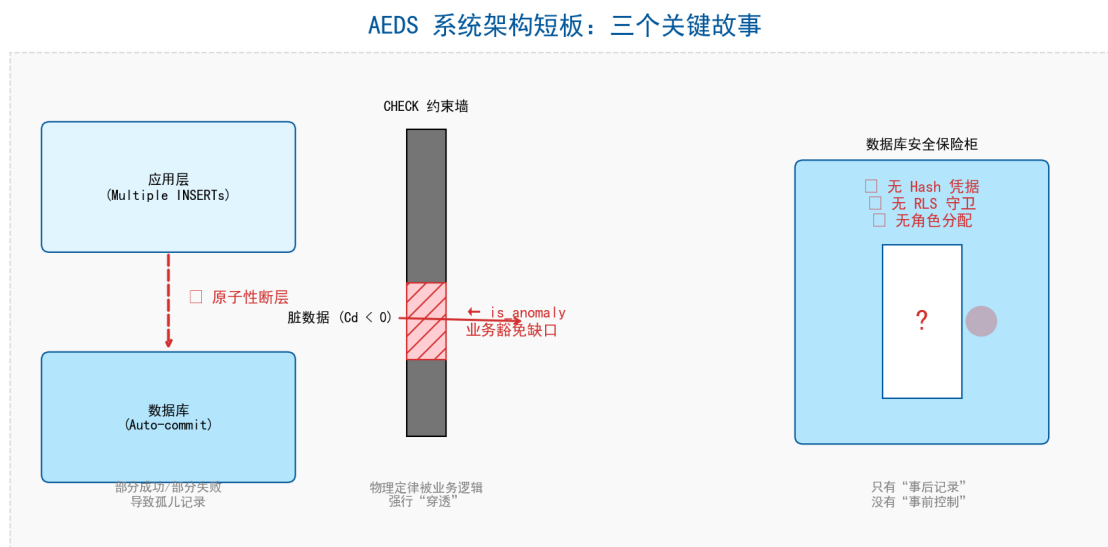


图 1: AEDS 系统架构中的三个关键短板：故事化呈现

6 总结与课程反思

三点不足的课程知识点对应

不足	涉及课程核心章节	问题性质
事务落地断层	事务处理 & 并发控制	理论与实践脱节
完整性约束妥协	数据完整性 & 约束设计	设计选择有瑕疵
安全性体系缺失	数据库安全性	核心维度系统性空白

从大二学生的角度审视，AEDS 项目的工程完成度较高（80% 达标率），Web 前端实现了丰富的交互功能，TestLab 实验套件覆盖了索引、事务和高级对象的全套验证。然而，数据库原理课程的考核核心并非”功能做得多的少”，而是”数据库理论是否真正落地”。以上三点不足共同揭示了一个深层问题：项目在”把实验结论转化为生产代码”这一环节存在断层——事务机制在 SQL 脚本中被验证了，但 Web 代码中没有用上；约束概念理解了，但设计时引入了业务层面的妥协；user 表建了，但安全性三个字始终停留在”做记录”而非”做控制”的层面。

这三点不足为后续版本改进提供了明确的技术锚点，也是数据库课程从”会用”到”会用对”这一关键跨越的真实写照。